Typography, Layout & Composition

Workshop 1 — Module 5

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

Module Roadmap

1. Typographic hierarchy: title, subtitle, body, caption
2. Font selection for data visualization
3. Whitespace, margins, and breathing room
4. Grid systems and multi-panel composition
5. Direct labelling, annotation, and export formats

Learning Outcome
You will be able to construct professional chart layouts with clear typographic hierarchy, purposeful whitespace, and publication-ready export settings.

Typographic Hierarchy

The Five Levels of Chart Typography

Typographic Hierarchy in Data Visualization

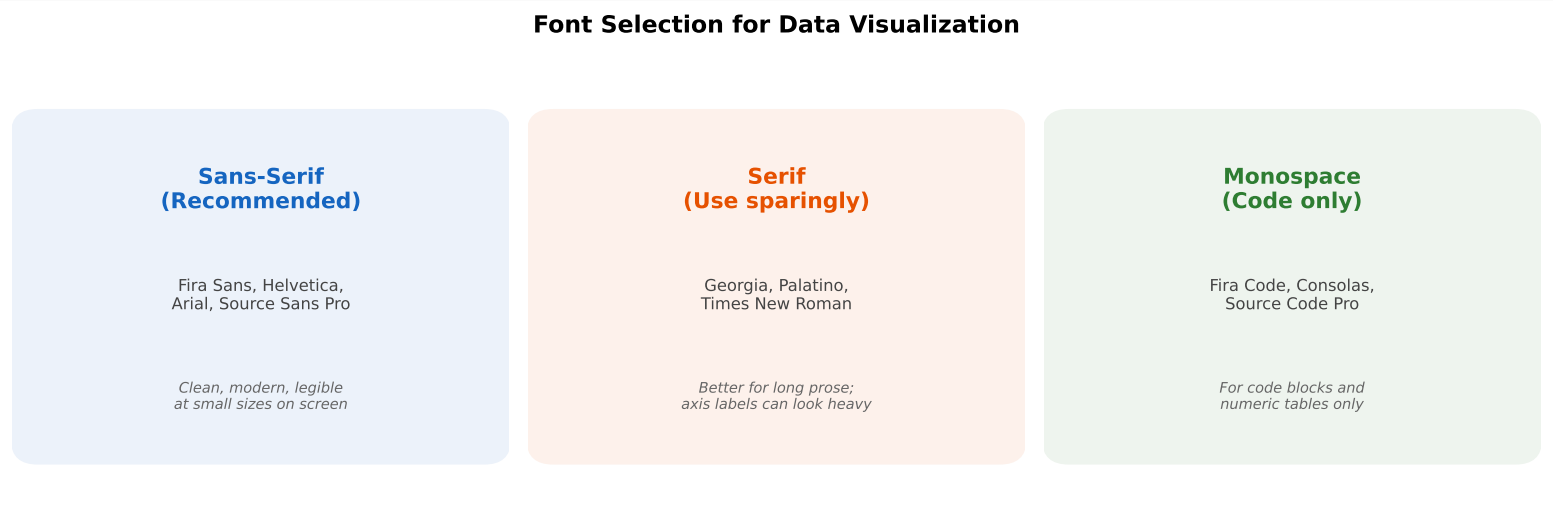
Chart Title: GDP Growth by Region	18pt Bold
Subtitle: Quarterly data, 2020-2024, seasonally adjusted	12pt Regular
Body Text / Axis Labels: 10-11pt, regular weight, high contrast against background	11pt Regular
Annotation: Direct labels on data points, 8-9pt, can use colour to match data series	9pt Regular
Caption / Source: Source: World Bank (2024). Note: excludes tax havens.	8pt Italic

Sizing Rules

Element	Weight	Size	Purpose
Title	Bold	16–20 pt	The "headline" of the chart
Subtitle	Regular	11–13 pt	Context, time range, units
Axis labels	Regular	10–11 pt	Variable names
Tick labels	Regular	8–10 pt	Scale values
Annotation	Regular	8–9 pt	Direct labels on data
Caption	Italic	7–8 pt	Source, notes, caveats

Pro-Tip
Maintain a **minimum 1.2× ratio** between adjacent hierarchy levels. If your title is 16pt, the subtitle should be $\leq$ 13pt and axis labels $\leq$ 11pt.

Font Selection



Data Insight

Sans-serif typefaces (Fira Sans, Helvetica, Arial) render crisply at small sizes on screens and are the default choice for all chart text. Reserve serif for reports; monospace for code and aligned numbers.

Setting Fonts in R

```r
library(ggplot2)

# Global font via theme
ggplot(df, aes(x, y)) +
  geom_point() +
  theme_minimal(
    base_size = 11,
    base_family = "Fira Sans"
  ) +
  theme(
    plot.title = element_text(
      size = 16,
      face = "bold",
      margin = margin(b = 8)),
    plot.subtitle = element_text(
      size = 11,
      color = "grey50"),
    plot.caption = element_text(
      size = 8,
      face = "italic",
      hjust = 0)
  ) +
  labs(
    title = "Main Title",
    subtitle = "Context line",
    caption = "Source: ...")
```

R font workflow:

- `base_family` sets the default for all text
- `element_text()` overrides individual elements
- `showtext::font_add_google()` loads Google Fonts
- `ragg` package renders custom fonts in PNG/PDF

Setting Fonts in Python

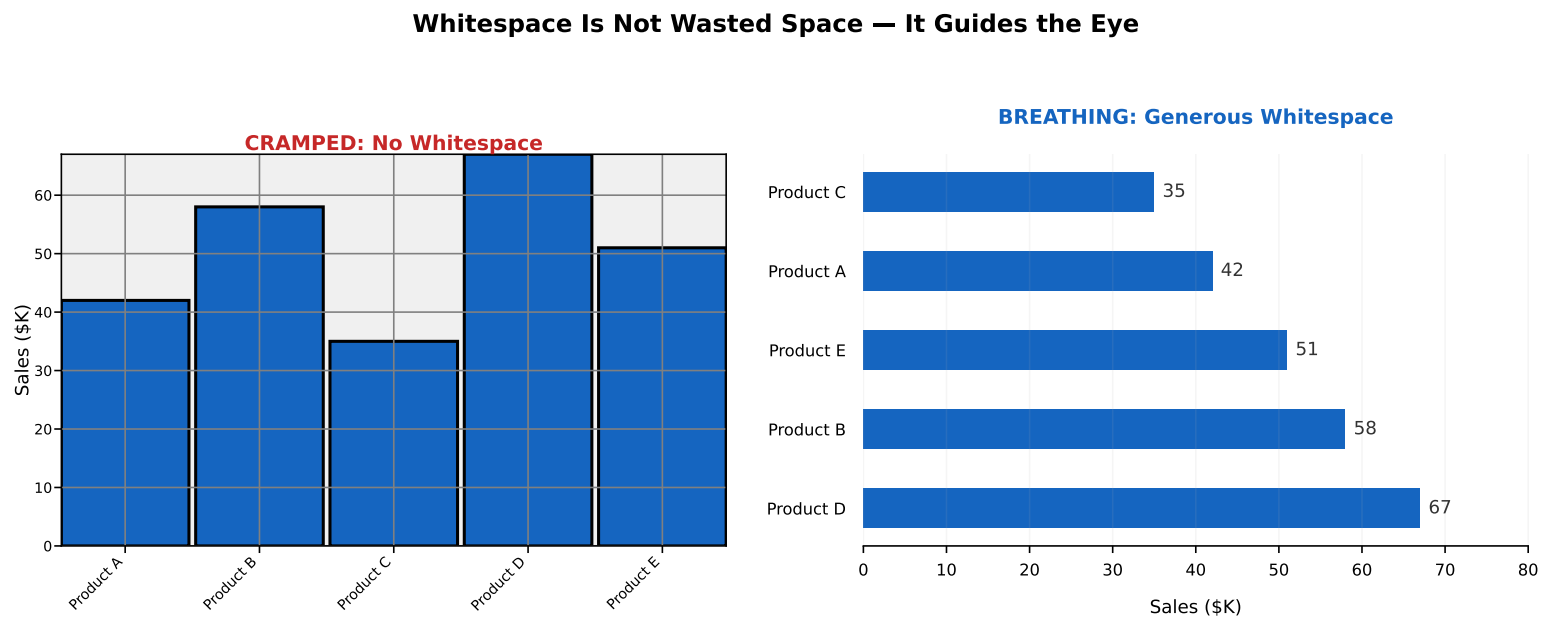
```python
import matplotlib.pyplot as plt

# Global rcParams
plt.rcParams.update({
    "font.family": "sans-serif",
    "font.sans-serif": [
        "Fira Sans", "Helvetica",
        "Arial"],
    "font.size": 11,
    "axes.titlesize": 14,
    "axes.titleweight": "bold",
    "axes.labelsize": 11,
    "xtick.labelsize": 9,
    "ytick.labelsize": 9,
})

fig, ax = plt.subplots()
ax.set_title("Main Title",
    fontsize=16, fontweight="bold")
# Subtitle via fig.text or
# ax.text below title
fig.text(0.5, 0.92,
    "Context line",
    ha="center", fontsize=11,
    color="grey")
```

Python font workflow:

- `rcParams` sets global defaults for all plots
- `fontweight`, `fontsize` override per-element
- For Google Fonts: download `.ttf` and register via `font_manager.fontManager.ad`
- `fig.text()` places text in figure coordinates

Whitespace & Margins

Whitespace Is Structure, Not Waste



Margin Guidelines

Internal margins:

- Title → plot area: 8–12 pt
- Plot area → axis labels: 6–8 pt
- Between facet panels: 8–10 pt
- Legend → plot edge: 6 pt

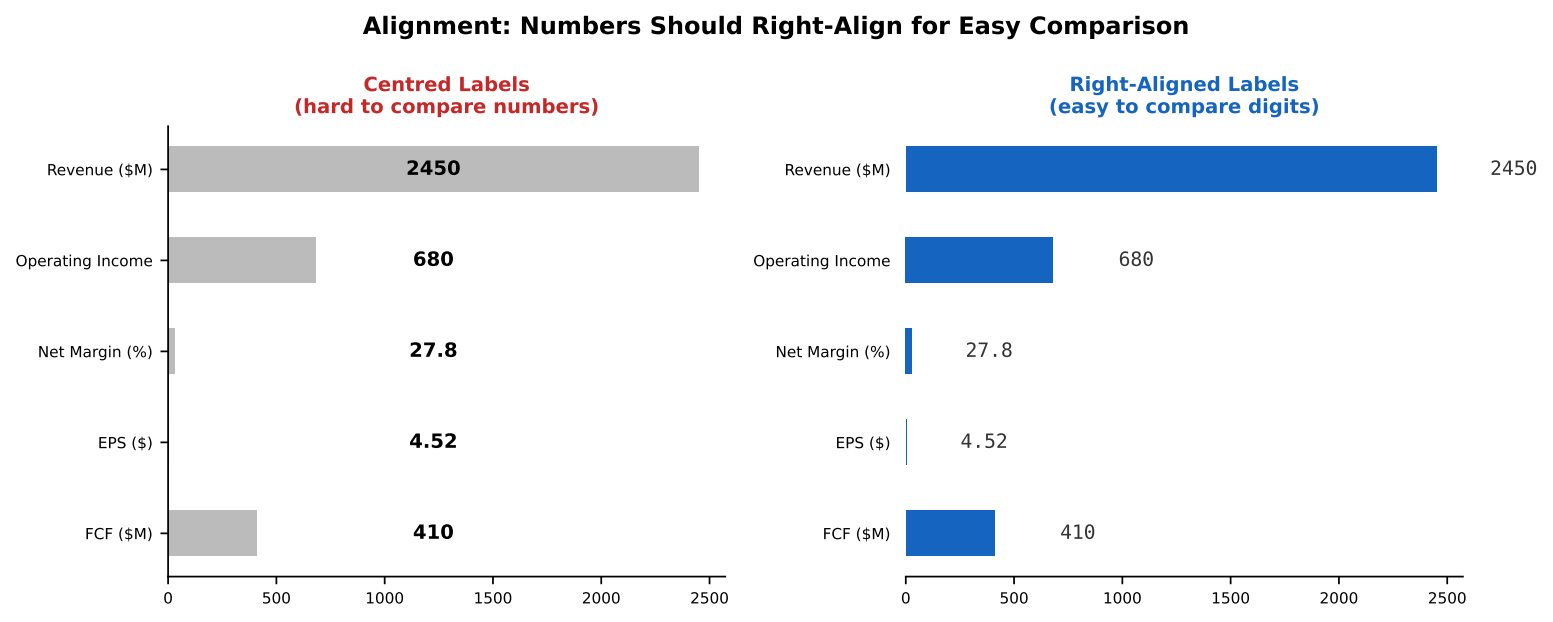
External margins:

- Document margin: 0.5–1.0 inch
- Chart inset from text: 12–18 pt
- Between stacked charts: 18–24 pt
- Caption below chart: 4–6 pt

Pro-Tip
In R use `theme(plot.margin = margin(t, r, b, l))`. In Python use `plt.tight_layout(pad=2.0)` or `plt.subplots_adjust()`.

Alignment & Direct Labelling

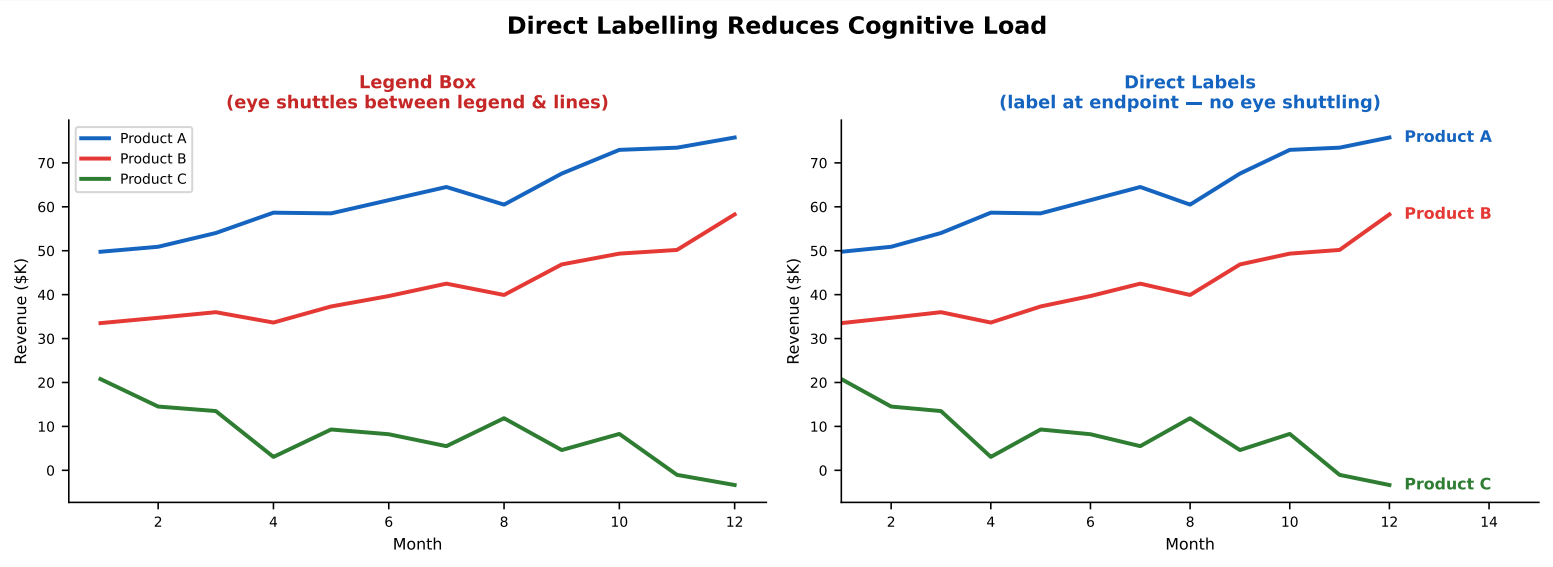
Alignment: Numbers Right-Align



Data Insight

Right-aligning numeric labels on the same column ensures that the digits align vertically, making magnitude comparison effortless. Centred numbers scatter the digits, forcing serial reading.

Direct Labelling vs Legend



Pro-Tip

A legend forces the eye to shuttle between the chart and the legend box. Direct labels at line endpoints (or on bars) eliminate this cognitive overhead. Use `ggrepel` in R or `adjustText` in Python to prevent label overlap.

Direct Labelling in R

```r
library(ggplot2)
library(ggrepel)

ggplot(df, aes(x = month,
               y = revenue,
               color = product)) +
  geom_line(linewidth = 1) +

  # Direct label at last point
  geom_text_repel(
    data = df |>
      filter(month == max(month)),
    aes(label = product),
    nudge_x = 0.5,
    size = 3,
    fontface = "bold",
    direction = "y") +

  # Remove legend (redundant)
  theme_minimal() +
  theme(legend.position = "none") +
  labs(x = "Month",
       y = "Revenue ($K)")
```

`ggrepel` **features:**

- Automatic collision avoidance
- Connector lines from label to data point
- `direction = "y"` constrains repulsion to vertical axis
- Works with `geom_text_repel` and `geom_label_repel`

Direct Labelling in Python

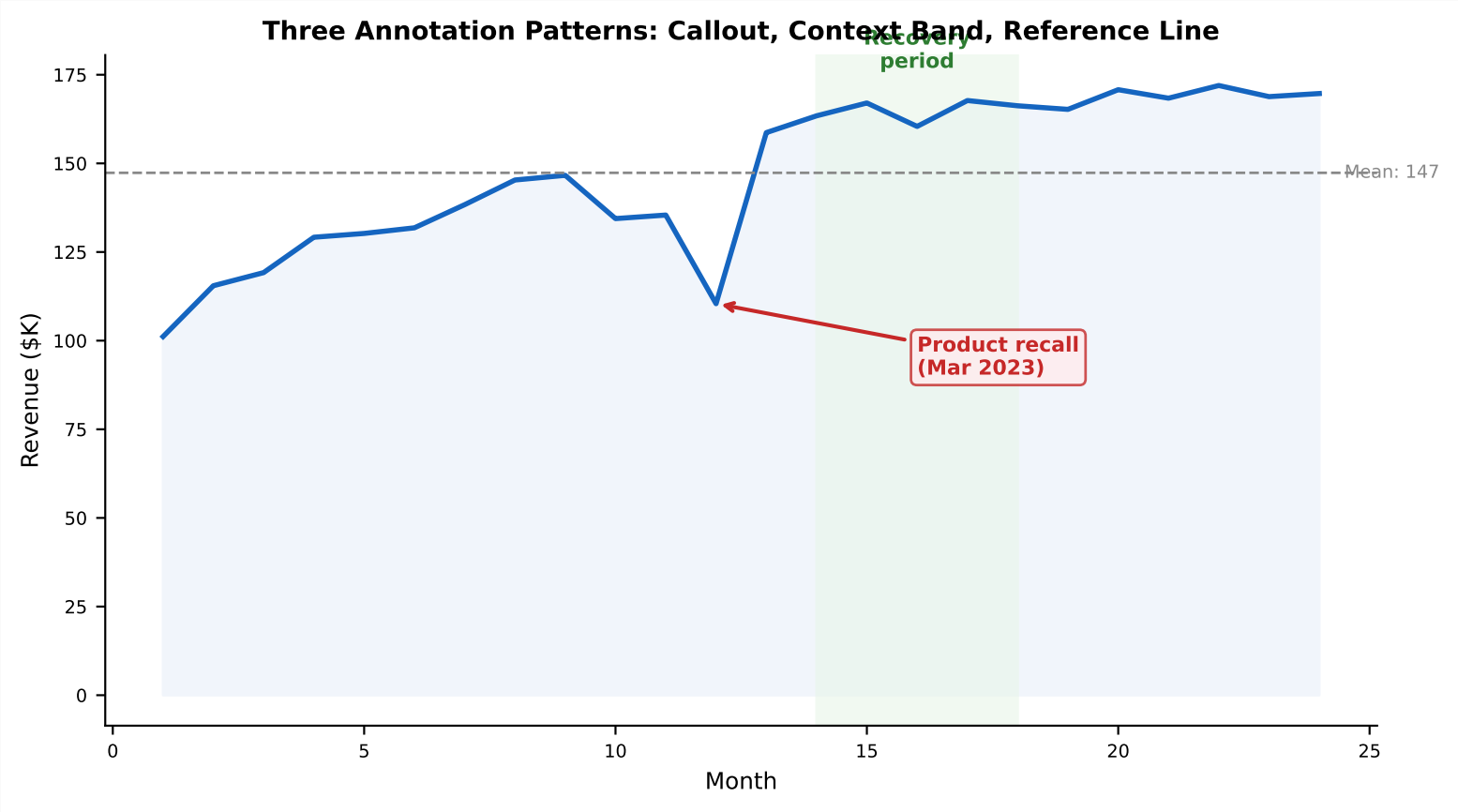
```python
import matplotlib.pyplot as plt
# pip install adjustText
from adjustText import adjust_text

fig, ax = plt.subplots()

for name, data in series.items():
    ax.plot(months, data,
        color=colors[name], lw=2)

    # Direct label at endpoint
    ax.text(
        months[-1] + 0.3,
        data[-1],
        name,
        fontsize=8,
        fontweight="bold",
        color=colors[name],
        va="center")

# Auto-adjust overlapping labels
# texts = [ax.text(...) for ...]
# adjust_text(texts)

ax.set_xlim(1, max(months) + 3)
```

Python pattern:

1. Place labels at line endpoints via `ax.text()`
2. Extend `xlim` to accommodate label width
3. Use `adjustText` for automatic collision avoidance
4. Match label colour to line colour for visual linking

Annotation Patterns

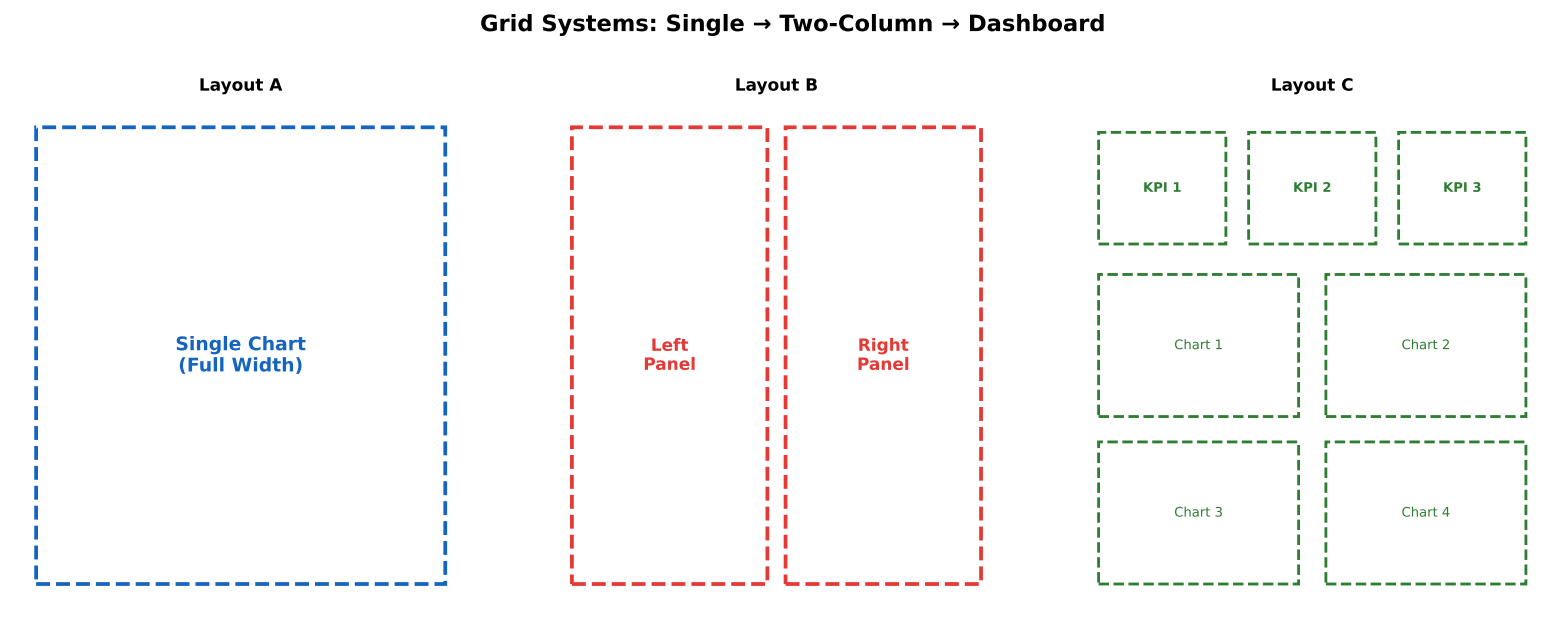
Three Annotation Patterns



- **Callout**: arrow + text box pointing to a specific event
- **Context band**: shaded region highlighting a time period
- **Reference line**: dashed horizontal for mean / target / benchmark

Grid Systems & Composition

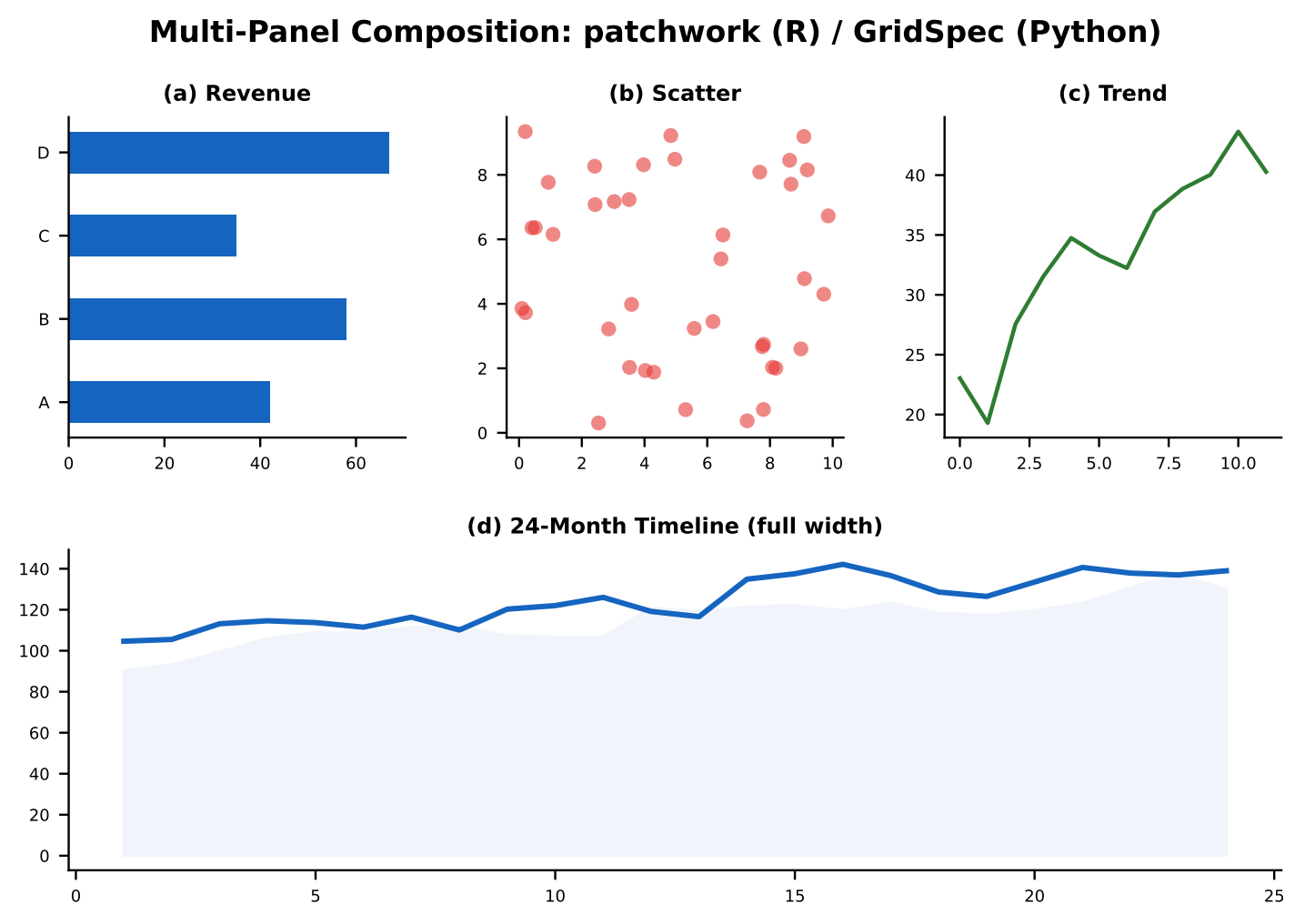
Layout Patterns



Data Insight

Single-chart layouts suit presentation slides. Two-column layouts support comparison (before/after, R vs Python). Dashboard grids (KPI row + chart grid) are standard in analytical reports.

Multi-Panel Composition



Multi-Panel in R: patchwork

```r
library(patchwork)

# Compose with operators
# | = side by side
# / = stacked
layout <- (p1 | p2 | p3) / p4

# Annotate the composite
layout +
  plot_annotation(
    title = "Quarterly Report",
    subtitle = "Revenue metrics",
    tag_levels = "a"  # (a),(b)...
  ) +
  plot_layout(
    heights = c(1, 1.5)  # bottom
                          # panel
                                wider
  )

# Alternative: cowplot
# library(cowplot)
# plot_grid(p1, p2, p3, p4,
#   ncol = 2, labels = "AUTO")
```

patchwork operators:

- `p1 | p2`: side by side
- `p1 / p2`: stacked
- `(p1 | p2) / p3`: row + row
- `plot_layout(widths=)`: control relative panel sizes
- `tag_levels`: auto-label panels (a), (b), (c)...

Multi-Panel in Python: GridSpec

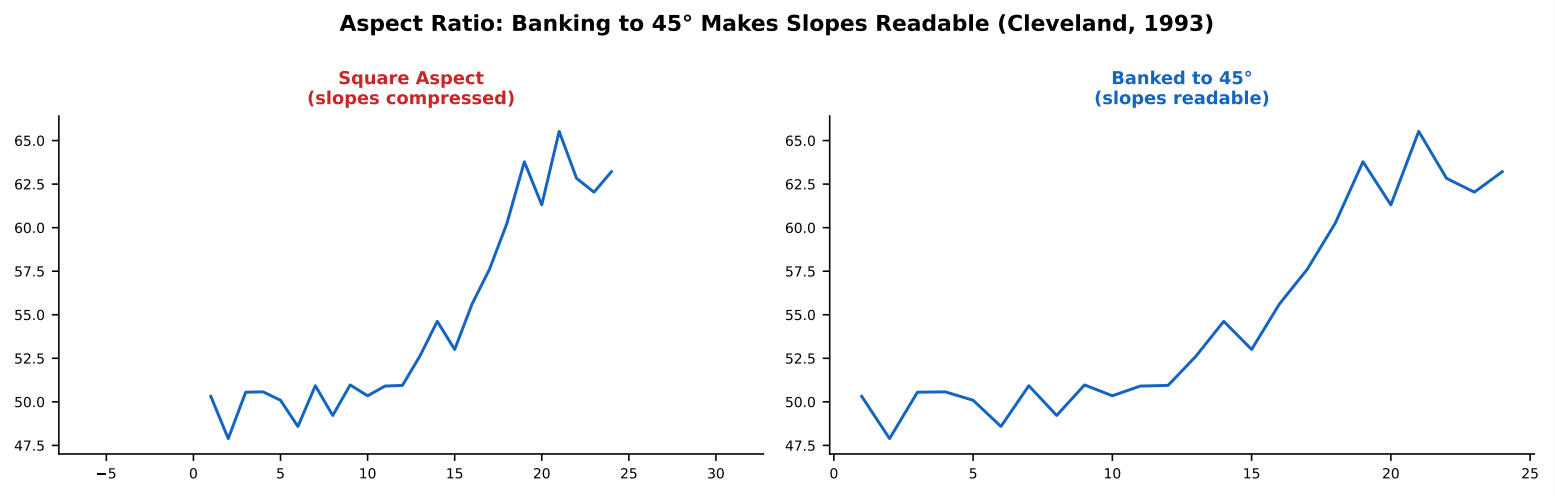
```python
from matplotlib.gridspec import (
    GridSpec)

fig = plt.figure(figsize=(8, 5))
gs = GridSpec(2, 3, figure=fig,
    hspace=0.35, wspace=0.3)

# Top row: 3 equal panels
ax1 = fig.add_subplot(gs[0, 0])
ax2 = fig.add_subplot(gs[0, 1])
ax3 = fig.add_subplot(gs[0, 2])

# Bottom: full-width panel
ax4 = fig.add_subplot(gs[1, :])

# Panel labels
for i, ax in enumerate(
    [ax1, ax2, ax3, ax4]):
    ax.text(0.02, 0.95,
        f"({chr(97+i)})",
        transform=ax.transAxes,
        fontsize=9,
        fontweight="bold",
        va="top")
```

GridSpec features:

- Arbitrary row/column spans via slice notation (`gs[1, :]`)
- `hspace`, `wspace` control inter-panel margins
- `height_ratios`, `width_ratios` for unequal panels
- `fig.add_subplot()` places axes on the grid

Aspect Ratio & Export

Aspect Ratio: Banking to 45 Degrees



Data Insight

Cleveland (1993) showed that line chart slopes are most accurately perceived when the average absolute slope is $\sim 45^\circ$. A square aspect ratio compresses temporal trends; a wider panel "banks" the slopes into the readable range.

Export Format Guide

Export Format Decision Guide

Format	Type	Best For	Resolution
PDF	Vector	Publication, print	Infinite
SVG	Vector	Web, interactive	Infinite
PNG	Raster	Web, presentations	≥150 dpi
JPEG	Raster (lossy)	Photos only	≥150 dpi
TIFF	Raster	Journal submission	≥300 dpi

Pro-Tip

Always export charts as **PDF or SVG** (vector) for publications and presentations. Use PNG at $\geq$ 150 dpi only when vector is not supported. Never use JPEG for charts — lossy compression creates artefacts around sharp edges and text.

Export in R

```r
# Vector (publication)
ggsave("chart.pdf",
  plot = p,
  width = 7,    # inches
  height = 5,
  device = cairo_pdf)

# Vector (web)
ggsave("chart.svg",
  plot = p,
  width = 7, height = 5)

# Raster (presentation)
ggsave("chart.png",
  plot = p,
  width = 7, height = 5,
  dpi = 300)
```

```python
# Vector (publication)
fig.savefig("chart.pdf",
    bbox_inches="tight")

# Vector (web)
fig.savefig("chart.svg",
    bbox_inches="tight")

# Raster (presentation)
fig.savefig("chart.png",
    dpi=300,
    bbox_inches="tight")

# NEVER: fig.savefig("chart.jpg")
# JPEG = lossy artefacts
```

Synthesis

Key Takeaways

1. **Typographic hierarchy** (title > subtitle > body > annotation > caption) guides the reader's eye through the chart. Maintain a 1.2× size ratio between levels.
2. **Sans-serif fonts** are the default for charts. Use `base_family` (R) or `rcParams` (Python).
3. **Whitespace** is structural, not decorative. Generous margins reduce cognitive load.
4. **Direct labels** at data points eliminate the legend-shuttle problem. Use `ggrepel` / `adjustText`.
5. **Multi-panel composition**: `patchwork` (R) and `GridSpec` (Python) support arbitrary layouts.
6. **Export as vector** (PDF/SVG) for print; PNG at $\geq$ 300 dpi for raster contexts. Never JPEG for charts.

Essential Readings for This Module

- Butterick, M. (2015). *Practical Typography*. `https://practicaltypography.com`
- Cleveland, W. S. (1993). "Banking to 45°." In: *Visualizing Data*, Hobart Press.
- Schwabish, J. (2021). *Better Data Visualizations*, Chapters 3–4 (Layout and Labelling).
- Wilke, C. O. (2019). *Fundamentals of Data Visualization*, Chapters 22–23 (Titles, Captions, Tables).

Next Module: The Design Process (W01-M06)